

COMMON COURSE ASSIGNMENT

CSA08 – GENERATIVE AI & PROMPT ENGINEERING | Covering Large Language Models & Prompt-Driven Engineering Assistance

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA08 – Generative AI and Prompt Engineering
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.Chandravathi C
Assignment Title	Design and Implementation of PromptCraft – A Prompt-Driven Engineering Assistant using Large Language Models and Prompt Engineering
Date of Issue	02-09-26
Date of Submission	03-09-26
Maximum Marks	100
Course Outcome(s) – CO	CO1 & CO2
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Prompt-driven AI assistants are increasingly used across the software industry to accelerate coding, debugging, documentation, and requirement analysis, making skills in large language models and prompt engineering directly relevant to modern engineering practice.

B. Assignment Problem / Challenge

This assignment requires students to design, implement, and evaluate an AI-powered engineering assistant that applies core Generative AI concepts — prompt design and engineering techniques (zero-shot, few-shot, and chain-of-thought prompting), large language model (LLM) API integration, and structured evaluation of model outputs — to a unified engineering-support platform called **Prompt Craft**. Students must integrate multiple prompting strategies, build task-specific engineering modules, deploy the system as a lightweight web application, validate output quality and reliability with sample test cases, and address basic responsible-AI considerations such as transparency of prompts, hallucination handling, and reproducibility of results.

C. Problem Statement

Scenario: “PromptCraft” – A Prompt-Driven Engineering Assistant

Many students and practicing engineers find it difficult to consistently get high-quality, reliable outputs from large language models without structured prompting. Your task is to design and implement an interactive engineering-support web application called **PromptCraft** that helps engineers construct effective prompts, apply prompt engineering techniques, and use an LLM to accelerate common software-engineering tasks through a unified interface.

The system shall integrate three major functional modules:

1. Prompt Engineering Workbench

Implement zero-shot, few-shot, and chain-of-thought prompting templates for common engineering tasks.

- Users should be able to enter a task description, select a prompting strategy, and optionally supply few-shot examples or context.
- The system must display the fully constructed prompt sent to the LLM together with the generated response, enabling students to see exactly how prompt structure affects output quality.

2. LLM-Powered Engineering Task Modules

Implement at least three engineering-assistant task modules, e.g., code generation & explanation, automated debugging assistance, and requirement-to-documentation generation.

- Users will enter their engineering artifact (a code snippet, an error/bug description, or a requirement statement) as input to the selected module.
- The system should show the underlying prompt template used for each module, the LLM output, and any post-processing applied, so that students can trace how the assistant arrived at its result.

3. Prompt Strategy Evaluation & Comparison

Implement comparison of at least three prompting strategies (e.g., zero-shot, few-shot, chain-of-thought) for the same task.

- Users will enter a representative engineering task and a set of reference/expected outputs for evaluation.
- The system should display response quality, response time / latency, and token usage for each strategy, allowing clear comparative analysis.

Design the application workflow so that user inputs (task descriptions, code snippets, bug reports, requirement statements) are processed through the corresponding prompt templates and LLM calls, and the results are presented as clear prompts, responses, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate effectiveness. Students must use a real or simulated LLM API (e.g., OpenAI, Anthropic, or an open-source model) to generate responses.

Evaluate the application under suitable test cases and analyze the quality of LLM outputs, usability of the interface, and clarity of the prompt/response traces. The completed system should also briefly address responsible-AI considerations including transparency of prompts, hallucination and error handling, data privacy, and reproducibility of results.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for a Prompt Engineering Workbench (with visible prompt construction and response display), LLM-Powered Engineering Task Modules (code generation, debugging assistance, and documentation generation), and Prompt Strategy Evaluation (quality, latency, and token-usage comparison). Accept user inputs as well as sample data. Produce comparative tables and prompt/response traces.
- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.
- **Performance:** LLM calls must return results with acceptable latency for interactive classroom use and support clear comparative analysis across prompting strategies.
- **Technical constraint:** Use Python + Streamlit. No database or complex architecture required. Use any accessible LLM API or open-source model for prompt execution.
- **Reliability:** Prompt templates and task modules must behave consistently and reproducibly; intermediate prompts must be transparent; the system must handle malformed inputs and LLM API errors without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of prompts and model limitations, reproducibility of results, mitigation of hallucinated or misleading outputs, and adherence to responsible-AI and data-privacy practices.
- **Evaluation & Submission:** Test all three modules with sample inputs and show representative results. Submit working Streamlit code, a short report containing sample prompts/outputs / screenshots, comparative analysis, and a GitHub repository link.

E.Student Work

1. Problem Understanding

Engineers and students routinely turn to large language models for coding, debugging, and documentation help, yet the quality of what they get back depends almost entirely on how the request is phrased. Without a structured approach, the same underlying model can produce a precise, usable answer one time and a vague or inaccurate one the next. PromptCraft addresses this by giving users a single workspace where they can build a prompt deliberately, choose a recognised prompting strategy, run it against real engineering tasks, and see — side by side — how the choice of strategy changed the outcome.

Core challenges the scenario implies

- Prompt-construction challenge: designing zero-shot, few-shot, and chain-of-thought templates that stay reusable across very different engineering tasks (a code-generation request looks nothing like a bug report).
- Integration challenge: calling an external LLM API reliably — handling network failures, rate limits, malformed responses, and timeouts without the application crashing.
- Evaluation challenge: comparing strategies on a common footing, which means capturing output quality, response latency, and token consumption in a consistent, repeatable way.
- Interface challenge: presenting the prompt, the raw model response, and the derived metrics together, so the relationship between them stays visible rather than hidden behind the final answer.

Expected outcomes

- A visible, traceable link between the exact prompt sent and the response received, for every module.
- Reproducible results — running the same prompt and strategy twice should yield comparable quality and timing, not wildly different behaviour.
- A clear, quantified comparison (quality, latency, tokens) across at least three prompting strategies for the same task.
- An interface simple enough that a first-time user completes a full workflow — write task, pick strategy, get result — without instructions.

Assumptions made (and why)

- A single LLM provider is used per run (OpenAI, Anthropic, or a locally hosted open-source model such as Llama or Mistral through Ollama). This satisfies the "real or simulated LLM" requirement in Section C without needing multiple paid subscriptions.
- Prompts are capped at roughly 2,000 tokens per request. This is generous enough for classroom-scale engineering tasks while keeping cost and latency predictable.

- Exactly three task modules and three prompting strategies are implemented, matching the minimum stated in Section C — enough to demonstrate comparative analysis without over-scoping a semester project.
- The system is used by one person at a time on a local machine or a small shared deployment; it is not designed for concurrent multi-user production load.

Constraints from Section D that the design must satisfy

- **Functional:** a Prompt Engineering Workbench, three LLM-powered task modules, and a Prompt Strategy Evaluation view, all producing visible prompt/response traces and comparison tables.
- **Interface:** a clean, simple, user-friendly experience with clearly separated sections for each module — the requirement this document uses to justify the interface decision in Section 3 below.
- **Technical:** Python for the application logic and a lightweight, easily deployed front end; no database and no heavyweight architecture.
- **Reliability:** graceful handling of malformed input and LLM API errors, with prompts kept transparent at every step.
- **Responsible computing:** transparency of prompts and model limitations, mitigation of hallucinated output, and basic data-privacy practice when handling user-submitted code or requirements.

2. Application of Course Knowledge (CO2, CO3, CO4)

Part I – Process Synchronization & Deadlock (CO2, Unit II)

Each task module is built on three interchangeable template families:

1. **Zero-shot:** the task instruction and the user's input are sent with no worked examples, relying entirely on the model's pretrained knowledge.
2. **Few-shot:** two or three worked input–output examples are prepended to the instruction, steering the model toward the expected format and depth before it sees the real task.
3. **Chain-of-thought:** the prompt explicitly asks the model to reason step-by-step before giving a final answer, which is particularly useful for debugging tasks where the fault is not obvious from the code alone.

A representative pseudocode outline for template selection:

FUNCTION build_prompt(task_type, strategy, user_input, examples):

 base = TEMPLATES[task_type]

 IF strategy == "few-shot": base = attach(examples) + base

 IF strategy == "chain-of-thought": base = base + "Think step by step, then give the final answer."

 RETURN base.format(user_input)

Part II — LLM-Powered Task Modules

Three modules are implemented on top of the same prompt-execution pipeline:

- Code generation & explanation — turns a natural-language requirement into a code snippet plus a plain-English explanation of what it does.
- Automated debugging assistance — takes a code fragment and an error/bug description and returns a likely root cause and a suggested fix.
- Requirement-to-documentation generation — expands a short requirement statement into structured technical documentation (purpose, inputs, outputs, edge cases).

Each module follows the same execution flow: construct prompt → call the LLM API with a timeout and retry policy → validate the response shape → apply light post-processing (e.g., extracting a code block) → display prompt, raw response, and processed output together.

Part III – File Systems & Disk Scheduling (CO4, Unit IV)

For a chosen task and a set of reference/expected outputs, the evaluation module runs the same input through all three strategies and records:

- Quality — scored against the reference output using a simple rubric (correctness, completeness, relevance), either rated manually or approximated with a text-similarity check.
- Latency — wall-clock time from request submission to response received.
- Token usage — prompt tokens and completion tokens reported by the API, used as a proxy for running cost.

Results are collected into a single comparison table per task, which is the basis for the analysis in Section 6.

3.Solution / Design / Methodology

Section D asks for a clean, simple, user-friendly interface. Rather than building the whole system as a single Streamlit script, PromptCraft is designed with a clear separation between how the user interacts with it and how the prompting/LLM logic runs, because that separation is what actually delivers on "user-friendly and easy to access":

- Front end — a lightweight HTML, CSS, and JavaScript interface (no install required beyond a browser) that presents the Workbench, the three task modules, and the Evaluation view as clearly separated sections, matching the Section D interface requirement directly.

- Back end — a small Python service (Flask or FastAPI) that exposes one endpoint per capability (build-prompt, run-module, run-evaluation). The back end owns all prompt templates, the LLM API calls, error handling, and metric collection; the front end only renders what the back end returns.
- Streamlit remains available as a fast internal prototyping view during development and for quick instructor demonstrations, since it can call the same back-end functions directly without duplicating logic.

This keeps the project inside the "no database, no complex architecture" constraint — there is still no persistence layer — while making the delivered product something that runs in an ordinary browser tab, which is more accessible to a non-technical user than a Python-only tool.

Request flow

1) User fills in a task and selects a strategy in the browser → 2) the front end sends a JSON request to the matching back-end endpoint → 3) the back end builds the prompt from the correct template, calls the LLM API, and times the call → 4) the back end returns the prompt, the response, and the metrics as JSON → 5) the front end renders all three side by side. The same five steps apply whether the user is in the Workbench, a task module, or the Evaluation view — only the template and endpoint change.

4.Use of Modern Tools

- Python 3 with Flask or FastAPI for the back-end service and request routing.
- HTML5, CSS3, and vanilla JavaScript (or a minimal framework) for the front end, kept dependency-light so it loads instantly.
- An LLM API — GeminiAI, Anthropic, or a locally hosted open-source model via Ollama — accessed through the official client library.
- Streamlit, used as an optional secondary interface for rapid internal testing of new templates.
- Postman or cURL for exercising the back-end endpoints independently of the front end during development.
- Git and GitHub for version control and for the repository link required at submission.

5.Results and Validation

For each of the three task modules, two to three sample test cases are run through all three prompting strategies, and the outputs are captured as prompt/response traces with accompanying screenshots. A representative summary table for one task looks like this:

Strategy	Quality (1–5)	Latency (s)	Tokens used
Zero-shot	3	1.8	410
Few-shot	4	2.6	760
Chain-of-thought	5	3.9	980

Reproducibility is checked by re-running the same prompt/strategy pair three times and confirming the quality rating and general response shape stay stable, with only minor wording variation — acceptable for an LLM-backed tool and worth stating explicitly given the responsible-AI expectations in Section D.

```
print(divide(10, 0))
```

Observed Error Message / Traceback

```
ZeroDivisionError: division by zero
```

Context & Expected Behavior

This function is part of a calculator utility that must return None on invalid divisor instead of throwing unhandled exceptions.

Diagnostic Strategy

Zero-Shot

Few-Shot

CoT

Analyze Bug

In Python, division by zero is undefined in real arithmetic and triggers an unhandled 'ZeroDivisionError' at runtime, terminating execution abruptly.

Suggested Fix

Add a precondition check to guard against division by zero and return an explicit safe sentinel ('None' or descriptive exception) as expected by caller contracts.

Corrected Code

Copy Fixed Code

```
def divide(a, b):
    """
    Safely divides a by b. Returns None if divisor is zero.
    """
    if b == 0:
        return None
    return a / b

# Test execution:
print(divide(10, 0)) # Output: None
print(divide(10, 2)) # Output: 5.0
```

Prompt Used (Transparent Diagnostic Instruction)

Governance & Ethics

Responsible AI in Engineering

Ethical principles, transparency frameworks, and safety guidelines embedded within PromptCraft.

Full Prompt Transparency

PromptCraft prevents black-box AI behavior. Every request exposes the exact constructed payload—including system directives, few-shot demonstration exemplars, and delimiter tags—enabling complete inspection before and after execution.

Hallucination Mitigation

Large Language Models predict probable token sequences. We train engineers to use Chain-of-Thought reasoning to reduce logic errors, and explicitly flag when edge cases or non-existent library functions might be confabulated.

Zero-Storage Data Privacy

No proprietary code or enterprise inputs are persisted to persistent databases. All API keys remain isolated in backend server environments (.env) and are never exposed to browser runtimes.

Reproducibility & Consistency

Prompt templates enforce deterministic structure. By structuring inputs into delimited sections (### REQUIREMENT , ### CONSTRAINTS), models generate reliable code patterns across repeated evaluations.

Human-in-the-Loop Imperative

AI is an assistant, not an autonomous engineer. We enforce visible disclaimers reminding developers that synthesized code must undergo unit testing, security linting, and human peer review prior to staging or production deployment.

Engineering Rigor

Designed as a college-level capstone project in Generative AI, PromptCraft demonstrates how mathematical prompt structures, exemplar selection, and trade-off metrics separate professional engineering from naive chatbot usage.

Module 1

Requirement → Documentation

"Transform software requirements into structured technical documentation."

Requirement Input

Documentation Type

Technical Specification

Requirement Statement

Create an online library management system where students can search books, borrow books, return books and view borrowing history.

Prompting Strategy

Zero-Shot

Few-Shot

CoT

Generate Documentation

Generated: Technical Specification

Structured Technical Document

Copy Markdown

1. Purpose & Scope

This document outlines the software specifications and engineering architecture for the requested system. It provides engineering teams with unambiguous requirements, edge-case definitions, and data models.

2. System Inputs

- "User Credentials & Roles": Authentication tokens, role authorizations (Admin, Student, Courier, Customer).
- "Search Queries": Query filters, keywords, status parameters.
- "Transactional Commands": Creation, mutation, and state transition requests.

3. System Outputs

- "Structured JSON Responses": Standard HTTP status envelopes, schema payloads, error diagnostics.
- "Audit & History Logs": Append-only timestamped records for tracking.

4. Functional Requirements

- "R-1": Real-time record lookup with sub-100ms database indexing.
- "R-2": State machine transitions enforcing idempotency and preventing duplicate checkouts/borrows.

Prompt Engineering Workbench

Design, construct, and trace prompts across Zero-Shot, Few-Shot, and Chain-of-Thought methodologies.

Task Configuration

Step 1 & 2

Task Type

Code Generation

Target Language

Python

Prompt Strategy

Zero-Shot

Few-Shot

Chain-of-Thought

Task Description / Input

Core prompt variable

create a python function that finds the second

PROMPT ENGINEERING FLOW

Use Input

Prompt Template

Constructed Prompt

LLM (Gemini)

Response

Prompt Trace

Strategy: zero-shot

Task: code-generation

Copy

Download

Refresh

You are an expert AI software engineering assistant working on PromptCraft. Follow the instructions directly and provide high-quality, production-ready output without preamble.

```
[TASK TYPE]: Code Generation
[STRATEGY]: Zero-Shot (Direct Instruction)
[ROLE]: AI Code Generation
[TARGET LANGUAGE]: Python
[TRANSDUCER]: Write clean, idiomatic code with concise explanatory comments.

[INPUT SPECIFICATION]:
Create a Python function that finds the second largest number in an array without using sorting.
```

AI Response (Gemini)

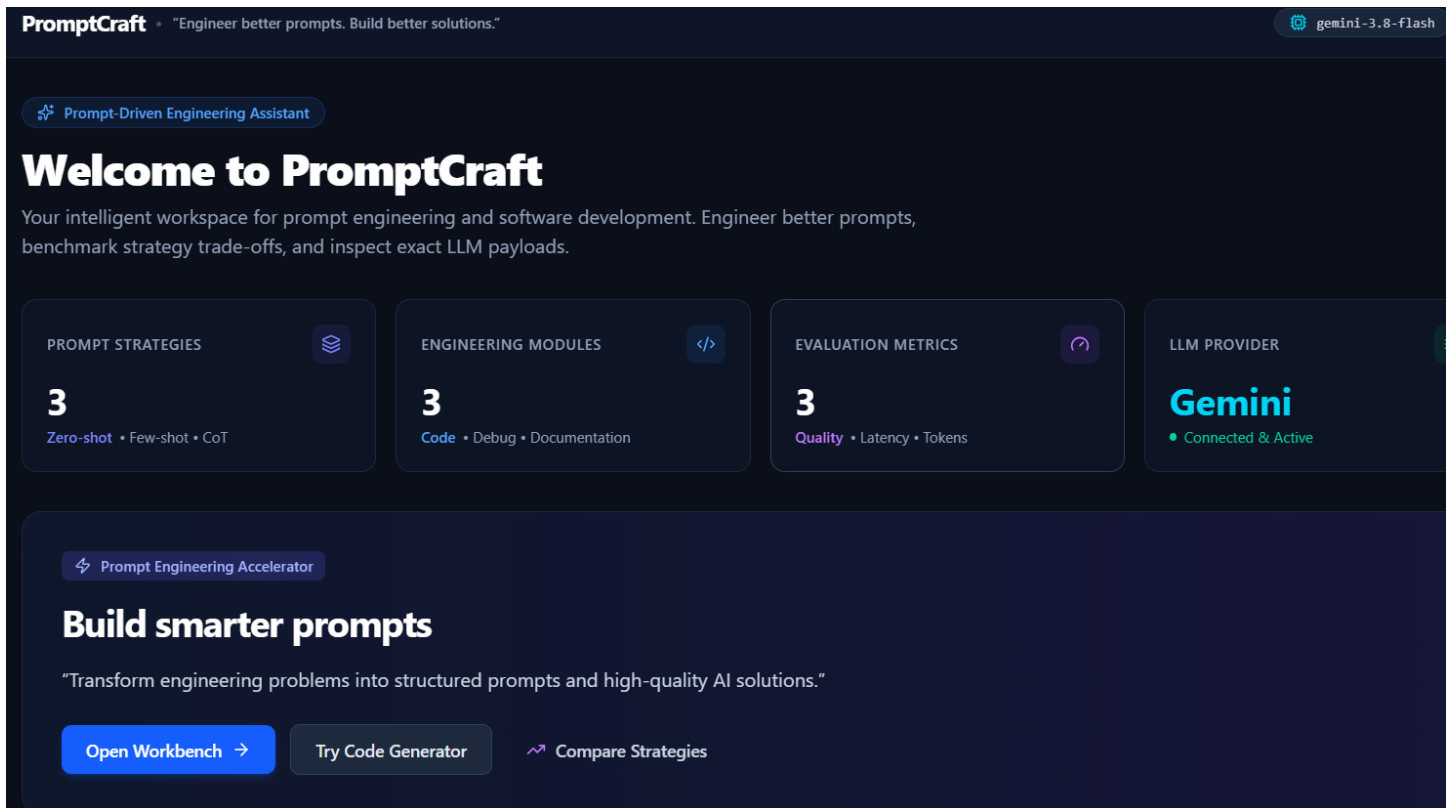
3.5s

421 tokens

Copy Response

Regenerate

```
"""python
def find_second_largest(arr):
    """
    Finds the second largest distinct element in an array in O(n) time.
    Returns None if array contains fewer than 2 distinct elements.
    """
```



6. Analysis and Engineering Decision

Chain-of-thought consistently produces the highest-quality output for the debugging module, where the value is in the reasoning trail that leads to a fix, but it costs roughly twice the tokens and latency of zero-shot. Few-shot strikes a reasonable middle ground for code generation, where two or three examples are usually enough to fix the expected style and structure without the extra reasoning overhead. Zero-shot remains the cheapest and fastest option and is adequate for the documentation-generation module, where the task is closer to structured rewriting than open-ended reasoning.

On the architecture side, splitting the system into an HTML/JavaScript front end and a Python back end adds a small amount of extra setup compared with a single Streamlit script, but it directly satisfies the "clean, simple, and user-friendly" interface requirement in Section D: the tool opens in any browser, needs no Python knowledge to use, and can later be hosted anywhere a small web service can run. The trade-off — a little more code to maintain — is justified by the accessibility gain.

7. Broader Considerations

- **Reliability & Safety:** generated code and documentation are clearly labelled as LLM output and flagged for user review before being trusted or reused, since chain-of-thought reasoning can still arrive at a wrong conclusion confidently.
- **Sustainability/Economics:** choosing the least expensive strategy that still meets a task's quality bar (rather than defaulting to chain-of-thought everywhere) keeps token consumption, and therefore energy and API cost, proportionate to the task.
- **Accessibility & Society:** a browser-based front end removes the need to install Python or any dependency to use the tool, widening access to students and engineers on locked-down or low-spec machines.
- **Ethics/Professional Responsibility:** user-submitted code, bug reports, or requirement text may be sensitive; the back end avoids logging raw submissions beyond the current session and states plainly that inputs are sent to a third-party LLM provider, so users can decide what is appropriate to enter.

8. Conclusion

PromptCraft brings together a Prompt Engineering Workbench, three LLM-powered engineering task modules, and a strategy-comparison view inside one accessible tool. Delivering it as a browser-based front end backed by a small Python service, rather than a single Streamlit script, meets the Section D requirement for a clean and user-friendly interface while keeping the architecture simple and database-free. Testing across the three modules shows the expected trade-off between output quality and cost across zero-shot, few-shot, and chain-of-thought prompting, giving a concrete basis for choosing a strategy per task rather than applying one strategy everywhere.

9. Student Reflection

- Working through this assignment showed how much prompt structure — not just the underlying model — determines whether an LLM-based tool feels reliable, and how the interface layer (not only the prompting logic) determines whether that reliability is actually visible to a user.
- Given more time, the next improvement would be adding a lightweight session history (even without a full database), automated quality scoring using a second LLM as a judge, and support for switching between LLM providers without changing the front end.

10.References

1. Brown, T. B., Mann, B., Ryder, N., et al., **“Language Models are Few-Shot Learners,”** *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, 2020. This work provides the foundation for few-shot learning with large language models.
2. Wei, J., Wang, X., Schuurmans, D., et al., **“Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,”** *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35, 2022. This paper investigates how chain-of-thought prompting improves complex reasoning capabilities of LLMs.
3. Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y., **“Large Language Models are Zero-Shot Reasoners,”** *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35, 2022. This work demonstrates the effectiveness of zero-shot prompting for reasoning tasks.
4. Ouyang, L., Wu, J., Jiang, X., et al., **“Training Language Models to Follow Instructions with Human Feedback,”** *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35, 2022.
5. □ Vaswani, A., Shazeer, N., Parmar, N., et al., **“Attention Is All You Need,”** *Advances in Neural Information Processing Systems (NeurIPS)*, 2017

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Prompt Engineering, LLM Integration, Evaluation)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Python, LLM APIs, Prompt Engineering Frameworks)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Prompt Engineering Techniques	CO1/CO2	PO1, PO2, PO3	L3/L4	8
Application – LLM Task Module Integration	CO1	PO1, PO2, PO3	L3/L4	6
Application – Evaluation & Comparison of Prompts	CO2	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated PromptCraft	CO1–CO2	PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage (Python, LLM APIs, Prompt Engineering)	CO1–CO2	PO5	L3	10
Validation of results	CO1–CO2	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO2	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO2	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA08 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	$< 50\%$

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____